JobSheet - Week 4 - Polymorphism

Nama: Zahra Azkiya Rahmani

NIM: 251524124

Kelas: 1D

Repo GitHub: https://github.com/yayyzea/1D_251524124_Teknik-Pemrograman-PR-.git

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W4_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W3_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Inheritance via Polymorphism (Another Type of Employee)

Soal Praktikum
Silahkan membaca soal pada file lampiran: 1-Polymorphism via Inheritance.pdf
Source Code

1. Firm.java

```
package w4_1_another_type_of_employee;

/**
 *
 * @author ZAHRA
 */

//*******************************************************************
// Firm.java Author: Lewis/Loftus
//
// Demonstrates polymorphism via inheritance.
//*******************************************************************

public class Firm{
    //-----------------------------------------------------------------
    // Creates a staff of employees for a firm and pays them.
    //-----------------------------------------------------------------
    public static void main (String[] args) {
        Staff personnel = new Staff();
        personnel.payday();
    }
}
```

2. Staff.java

```
package w4_1_another_type_of_employee;
```

```
/**
 *
 * @author ZAHRA
 */

//*******************************************************************
// Staff.java Author: Lewis/Loftus
//
// Represents the personnel staff of a particular business.
//*******************************************************************
public class Staff{
   StaffMember[] staffList;

   //-----------------------------------------------------------------
   // Sets up the list of staff members.
   //-----------------------------------------------------------------
   public Staff (){
      staffList = new StaffMember[8];
      staffList[0] = new Executive("Sam","123 Main Line","555-0469","123-45-6789",2423.07);
      staffList[1] = new Employee("Carla","456 Off Line","555-0101","987-65-4321",1246.15);
      staffList[2] = new Employee("Woody","789 Off Rocker","555-0000","010-20-3040",1169.23);
      staffList[3] = new Hourly("Diane","678 Fifth Ave.","555-0690","958-47-3625",10.55);
      staffList[4] = new Volunteer("Norm","987 Suds Blvd.","555-8374");
      staffList[5] = new Volunteer("Cliff","321 Duds Lane","555-7282");
      staffList[6] = new Commission("Stelle","100 Sales Rd.","555-1111","111-11-1111",6.25,0.20);
      staffList[7] = new Commission("Flins","200 Sales Ave.","555-2222","222-22-2222",9.75,0.15);

      ((Executive)staffList[0]).awardBonus(500.00);
      ((Hourly)staffList[3]).addHours(40);
      ((Commission)staffList[6]).addHours(35);
      ((Commission)staffList[6]).addSales(400);
      ((Commission)staffList[7]).addHours(40);
      ((Commission)staffList[7]).addSales(950);
   }

   //-----------------------------------------------------------------
   // Pays all staff members.
   //-----------------------------------------------------------------
   public void payday () {
      double amount;
      for (int count=0; count < staffList.length; count++){
         System.out.println (staffList[count]);
         amount = staffList[count].pay(); // polymorphic
         if (amount == 0.0)
            System.out.println ("Thanks!");
         else
            System.out.println ("Paid: " + amount);
            System.out.println ("-----------------------------------");
      }
   }
}
```

3. StaffMember.java

```
package w4_1_another_type_of_employee;

/**
 *
 * @author ZAHRA
 */

//*******************************************************************
// StaffMember.java Author: Lewis/Loftus
//
// Represents a generic staff member.
//*******************************************************************

abstract public class StaffMember{
   protected String name;
   protected String address;
   protected String phone;
   //-----------------------------------------------------------------
   // Sets up a staff member using the specified information.
   //-----------------------------------------------------------------
   public StaffMember (String eName, String eAddress, String ePhone){
      name = eName;
      address = eAddress;
      phone = ePhone;
   }

   //-----------------------------------------------------------------
   // Returns a string including the basic employee information.
   //-----------------------------------------------------------------
   public String toString(){
```

```
        String result = "Name: " + name + "\n";
        result += "Address: " + address + "\n";
        result += "Phone: " + phone;
        return result;
    }

    //-----------------------------------------------------------------
    // Derived classes must define the pay method for each type of
    // employee.
    //-----------------------------------------------------------------
    public abstract double pay();
}
```

4. Volunteer.java

```
package w4_1_another_type_of_employee;

/**
 *
 * @author ZAHRA
 */
//*******************************************************************
// Volunteer.java Author: Lewis/Loftus
//
// Represents a staff member that works as a volunteer.
//*******************************************************************

public class Volunteer extends StaffMember{
    //-----------------------------------------------------------------
    // Sets up a volunteer using the specified information.
    //-----------------------------------------------------------------
    public Volunteer (String eName, String eAddress, String ePhone){
        super (eName, eAddress, ePhone);
    }

    //-----------------------------------------------------------------
    // Returns a zero pay value for this volunteer.
    //-----------------------------------------------------------------
    public double pay(){
        return 0.0;
    }
}
```

5. Employee.java

```
package w4_1_another_type_of_employee;

/**
 *
 * @author ZAHRA
 */

//*******************************************************************
// Employee.java Author: Lewis/Loftus
//
// Represents a general paid employee.
//*******************************************************************

public class Employee extends StaffMember{
    protected String socialSecurityNumber;
    protected double payRate;

    //-----------------------------------------------------------------
    // Sets up an employee with the specified information.
    //-----------------------------------------------------------------
    public Employee (String eName, String eAddress, String ePhone, String socSecNumber, double rate){
        super (eName, eAddress, ePhone);
        socialSecurityNumber = socSecNumber;
        payRate = rate;
    }

    //-----------------------------------------------------------------
    // Returns information about an employee as a string.
    //-----------------------------------------------------------------
    public String toString(){
        String result = super.toString();
        result += "\nSocial Security Number: " + socialSecurityNumber;
        return result;
    }

    //-----------------------------------------------------------------
    // Returns the pay rate for this employee.
    //-----------------------------------------------------------------
    public double pay(){
        return payRate;
```

```
    }
}
```

6. Executive.java

```
package w4_1_another_type_of_employee;

/**
 *
 * @author ZAHRA
 */
//*******************************************************************
// Executive.java Author: Lewis/Loftus
//
// Represents an executive staff member, who can earn a bonus.
//*******************************************************************

public class Executive extends Employee{
    private double bonus;
    //-----------------------------------------------------------------
    // Sets up an executive with the specified information.
    //-----------------------------------------------------------------
    public Executive (String eName, String eAddress, String ePhone, String socSecNumber, double rate){
        super (eName, eAddress, ePhone, socSecNumber, rate);
        bonus = 0; // bonus has yet to be awarded
    }

    //-----------------------------------------------------------------
    // Awards the specified bonus to this executive.
    //-----------------------------------------------------------------
    public void awardBonus (double execBonus){
       bonus = execBonus;
    }

    //-----------------------------------------------------------------
    // Computes and returns the pay for an executive, which is the
    // regular employee payment plus a one-time bonus.
    //-----------------------------------------------------------------
    public double pay(){
       double payment = super.pay() + bonus;
       bonus = 0;
       return payment;
    }
}
```

7. Hourly.java

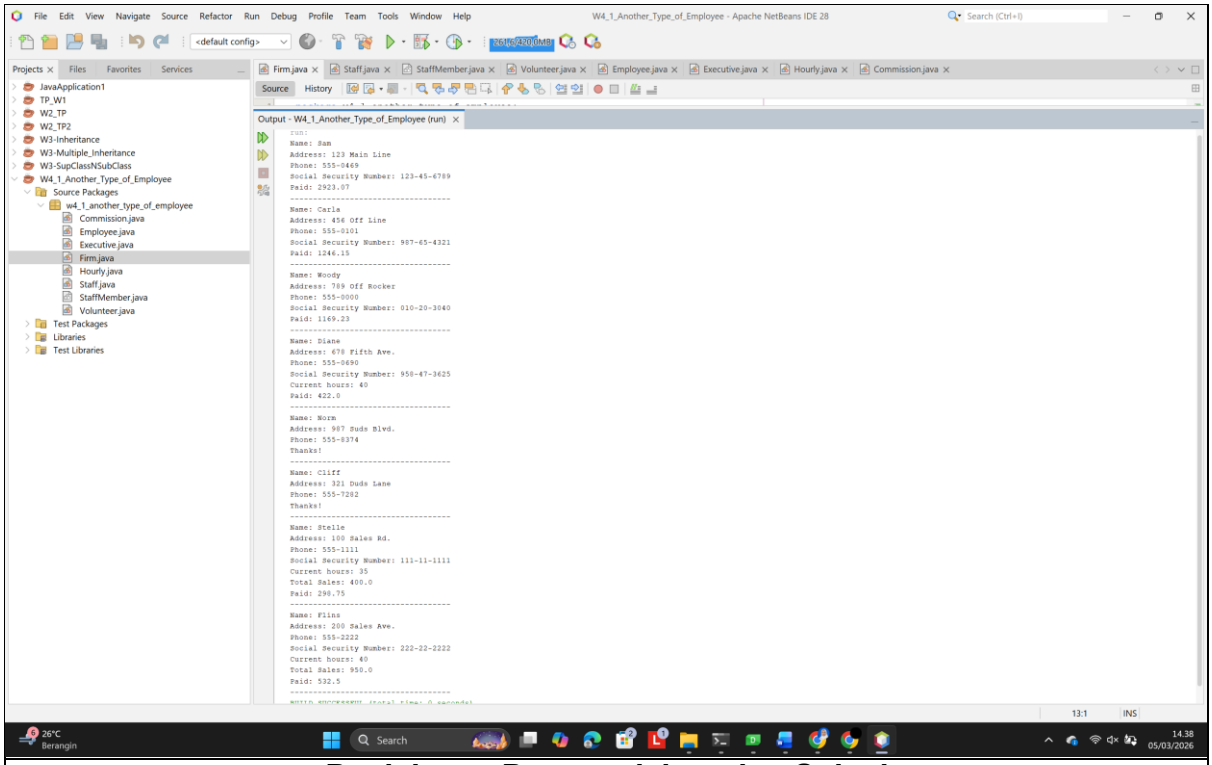
```
package w4_1_another_type_of_employee;

/**
 *
 * @author ZAHRA
 */

//*******************************************************************
// Hourly.java Author: Lewis/Loftus
//
// Represents an employee that gets paid by the hour.
//*******************************************************************
public class Hourly extends Employee{
    private int hoursWorked;
    //-----------------------------------------------------------------
    // Sets up this hourly employee using the specified information.
    //-----------------------------------------------------------------
    public Hourly (String eName, String eAddress, String ePhone, String socSecNumber, double rate){
       super (eName, eAddress, ePhone, socSecNumber, rate);
       hoursWorked = 0;
    }

    //-----------------------------------------------------------------
    // Adds the specified number of hours to this employee's
    // accumulated hours.
    //-----------------------------------------------------------------
    public void addHours (int moreHours){
        hoursWorked += moreHours;
    }

    //-----------------------------------------------------------------
    // Computes and returns the pay for this hourly employee.
    //-----------------------------------------------------------------
    public double pay() {
        double payment = payRate * hoursWorked;
        hoursWorked = 0;
        return payment;
    }
```

```
    //-----------------------------------------------------------------
    // Returns information about this hourly employee as a string.
    //-----------------------------------------------------------------
    public String toString() {
        String result = super.toString();
        result += "\nCurrent hours: " + hoursWorked;
        return result;
    }
}
```

8. Commission.java

```
package w4_1_another_type_of_employee;

/**
 *
 * @author ZAHRA
 */

// ******************************************************
// Commission.java
//
// Represents an hourly employee who also earns commission on sales.
// ******************************************************

public class Commission extends Hourly{
    private double totalSales;
    private double commissionRate;

    public Commission(String name, String address, String phone,String ssn, double rate, double commRate){
        super(name, address, phone, ssn, rate);
        this.commissionRate = commRate;
        this.totalSales = 0.0;
    }

    public void addSales(double sales) {
        totalSales += sales;
    }

    //-----------------------------------------------------------------
    // Returns the pay for this commission employe.
    //-----------------------------------------------------------------
    public double pay() {
        double payment = super.pay() + (totalSales * commissionRate);
        totalSales = 0.0;
        return payment;
    }

    //-----------------------------------------------------------------
    // Returns information about this commission employee as a string,
    // including inherited info and the current total sales figure.
    //-----------------------------------------------------------------
    public String toString() {
        return super.toString() + "\nTotal Sales: " + totalSales;
    }
}
```

Screenshot Hasil



Penjelasan Permasalahan dan Solusi

Permasalahan:

Sistem membutuhkan satu tipe karyawan baru yang sistem gajinya merupakan kombinasi antara upah per jam dan komisi dari hasil penjualan. Masalah utamanya adalah bagaimana mengintegrasikan atribut baru (totalSales dan commissionRate) ke dalam struktur karyawan tanpa merusak atau menulis ulang logika perhitungan jam kerja yang sudah ada. Selain itu, sistem pemanggilan gaji (payday) di kelas utama harus memproses tipe karyawan baru ini secara otomatis tanpa perlu mengubah logika perulangannya.

Solusi:

Masalah ini diselesaikan dengan menerapkan konsep Multilevel Inheritance. Kelas Commission dibuat sebagai subclass yang mewarisi kelas Hourly, yang mana Hourly sendiri mewarisi Employee.

- Pada Commission.java, konstruktor memanggil super(name, address, phone, ssn, rate) untuk menyerahkan inisialisasi data dasar pegawai kepada superclass, lalu dilanjutkan dengan menginisialisasi atribut spesifik commissionRate dan menyetel totalSales ke 0.0.
- Kalkulasi gaji diselesaikan dengan melakukan overriding pada method pay(). Di dalamnya, dipanggil super.pay() untuk mengeksekusi perhitungan upah jam kerja secara otomatis (sekaligus mereset hoursWorked di superclass). Hasil tersebut kemudian ditambahkan dengan kalkulasi komisi (totalSales * commissionRate). Setelahnya, totalSales direset menjadi 0.0.
- Pada Staff.java, kapasitas array staffList diperbesar menjadi 8 untuk menampung dua objek Commission baru (Stelle dan Flins). Saat method payday() melakukan perulangan dan memanggil staffList[count].pay(), program mendemonstrasikan Dynamic Binding. Meskipun referensinya bertipe umum (StaffMember), Java menunda pengikatan method hingga run time dan secara cerdas mengeksekusi method pay() yang paling spesifik milik objek Commission.

Dengan memanggil super.pay(), kita menghindari duplikasi kode. Solusi ini menjaga integritas enkapsulasi, di mana kelas induk (Hourly) tetap memegang kendali penuh untuk mereset variabel miliknya sendiri (hoursWorked), sementara kelas anak (Commission) hanya fokus menambahkan logika spesifiknya saja (menghitung dan mereset komisi).

Nama Teman Hal yang Dibantu (Opsional)

No. 2 Inheritance via Polymorphism (Painting Shapes)

Soal Praktikum
Silahkan membaca soal pada file lampiran: 2-Polymorphism via Inheritance.pdf
Source Code

1. **Shape.java**

```
package w4_2_inheritance_via_polymorphism_.painting_shapes;

/**
 *
 * @author ZAHRA
 */

//*******************************************************************
// Shape.java
//
// Represents a generic geometric shape. This is an abstract class
// that serves as the parent of all specific shape classes.
// Each derived class must implement the area() method.
//*******************************************************************

public abstract class Shape {
   protected String shapeName;

   //-----------------------------------------------------------------
   // Sets up the shape with the given name. Called by constructors
   // of derived classes via super().
   //-----------------------------------------------------------------
   public Shape(String shapeName) {
      this.shapeName = shapeName;
   }

   //-----------------------------------------------------------------
   // Returns the area of this shape.
   //-----------------------------------------------------------------
   public abstract double area();

   //-----------------------------------------------------------------
   // Returns the name of this shape as a string.
   //-----------------------------------------------------------------
   public String toString() {
      return shapeName;
   }
}
```

2. **Sphere.java**

```
package w4_2_inheritance_via_polymorphism_.painting_shapes;

/**
 *
 * @author ZAHRA
 */

//****************************************
// Sphere.java
//
// Represents a sphere.
//****************************************
public class Sphere extends Shape {
   private double radius; //radius in feet
```

```
    //-----------------------------------
    // Constructor: Sets up the sphere.
    //-----------------------------------
    public Sphere(double r) {
        super("Sphere");
        radius = r;
    }

    //-----------------------------------------
    // Returns the surface area of the sphere.
    //-----------------------------------------
    public double area() {
        return 4 * Math.PI * radius * radius;
    }

    //-----------------------------------
    // Returns the sphere as a String.
    //-----------------------------------
    public String toString() {
        return super.toString() + " of radius " + radius;
    }
}
```

3. Rectangle.java

```
package w4_2_inheritance_via_polymorphism_.painting_shapes;

/**
 *
 * @author ZAHRA
 */

//*******************************************************************
// Rectangle.java
//
// Represents a rectangle shape.
//*******************************************************************

public class Rectangle extends Shape {
    private double length, width;

    //-----------------------------------------------------------------
    // Sets up the rectangle with the specified length and width.
    // Passes the shape name "Rectangle" to the parent constructor.
    //-----------------------------------------------------------------
    public Rectangle(double l, double w) {
        super("Rectangle");
        length = l;
        width = w;
    }

    //-----------------------------------------------------------------
    // Returns the area of this rectangle, computed as length * width.
    //-----------------------------------------------------------------
    public double area() {
        return length * width;
    }

    //-----------------------------------------------------------------
    // Returns the rectangle as a string including its dimensions.
    //-----------------------------------------------------------------
    public String toString() {
        return super.toString() + " of length " + length + " and width " + width;
    }
}
```

4. Cylinder.java

```
package w4_2_inheritance_via_polymorphism_.painting_shapes;

/**
 *
 * @author ZAHRA
 */

//*******************************************************************
// Cylinder.java
//
// Represents a cylinder shape.
//*******************************************************************

public class Cylinder extends Shape {
```

```
    private double radius, height;

    //-----------------------------------------------------------------
    // Sets up the cylinder with the specified radius and height.
    // Passes the shape name "Cylinder" to the parent constructor.
    //-----------------------------------------------------------------
    public Cylinder(double r, double h) {
        super("Cylinder");
        radius = r;
        height = h;
    }

    //-----------------------------------------------------------------
    // Returns the lateral surface area of this cylinder,
    //-----------------------------------------------------------------
    public double area() {
        return Math.PI * radius * radius * height;
    }

    //-----------------------------------------------------------------
    // Returns the cylinder as a string including its dimensions.
    //-----------------------------------------------------------------
    public String toString() {
        return super.toString() + " of radius " + radius + " and height " + height;
    }
}
```

5. Paint.java

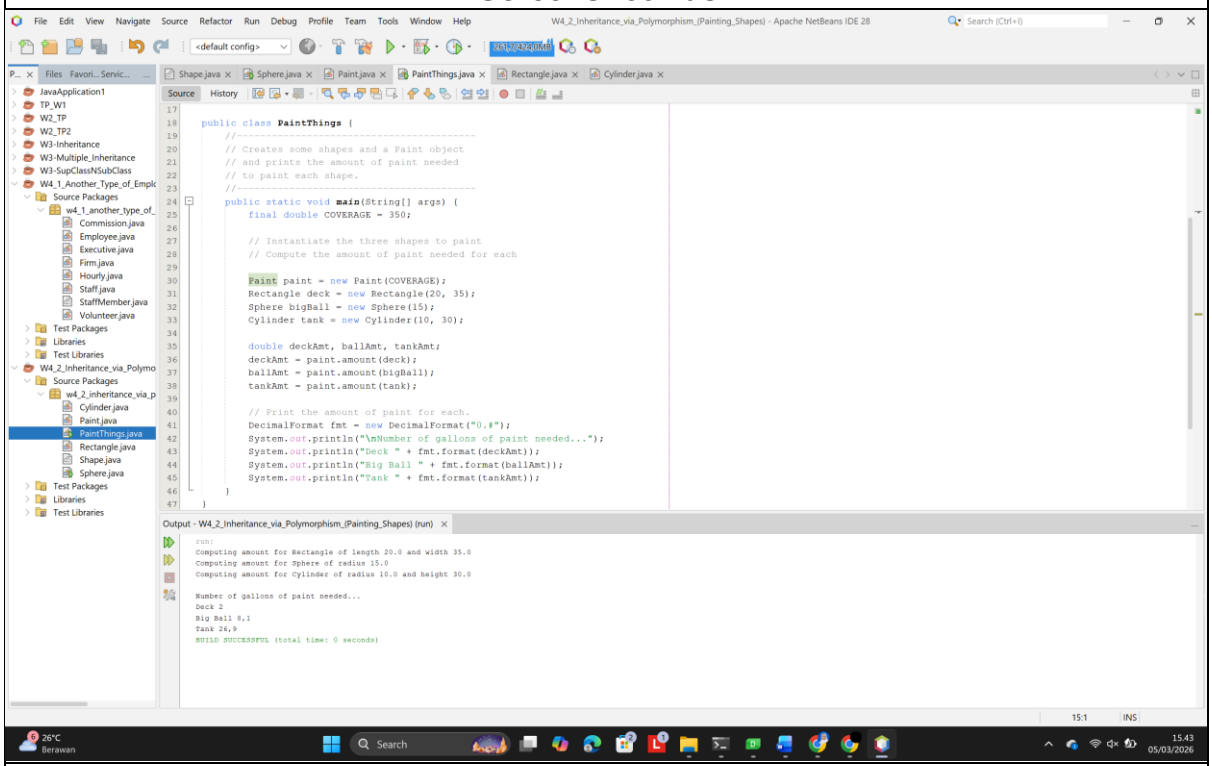
```
package w4_2_inheritance_via_polymorphism_.painting_shapes;

/**
 *
 * @author ZAHRA
 */

//*******************************************************
// Paint.java
//
// Represents a type of paint that has a fixed area
// covered by a gallon. All measurements are in feet.
//*******************************************************
public class Paint {
    private double coverage; //number of square feet per gallon

    //-----------------------------------------
    // Constructor: Sets up the paint object.
    //-----------------------------------------
    public Paint(double c) {
        coverage = c;
    }

    //-------------------------------------------------
    // Returns the amount of paint (number of gallons)
    // needed to paint the shape given as the parameter.
    //-------------------------------------------------
    public double amount(Shape s) {
        System.out.println("Computing amount for " + s);
        return s.area() / coverage;
    }
}
```

6. PaintThings.java

```
package w4_2_inheritance_via_polymorphism_.painting_shapes;

/**
 *
 * @author ZAHRA
 */

//*************************************************************
// PaintThings.java
//
// Computes the amount of paint needed to paint various
// things. Uses the amount method of the paint class which
// takes any Shape as a parameter.
//*************************************************************

import java.text.DecimalFormat;

public class PaintThings {
    //-----------------------------------------
```

```
  // Creates some shapes and a Paint object
  // and prints the amount of paint needed
  // to paint each shape.
  //----------------------------------------
  public static void main(String[] args) {
     final double COVERAGE = 350;

     // Instantiate the three shapes to paint
     // Compute the amount of paint needed for each

     Paint paint = new Paint(COVERAGE);
     Rectangle deck = new Rectangle(20, 35);
     Sphere bigBall = new Sphere(15);
     Cylinder tank = new Cylinder(10, 30);

     double deckAmt, ballAmt, tankAmt;
     deckAmt = paint.amount(deck);
     ballAmt = paint.amount(bigBall);
     tankAmt = paint.amount(tank);

     // Print the amount of paint for each.
     DecimalFormat fmt = new DecimalFormat("0.#");
     System.out.println("\nNumber of gallons of paint needed...");
     System.out.println("Deck " + fmt.format(deckAmt));
     System.out.println("Big Ball " + fmt.format(ballAmt));
     System.out.println("Tank " + fmt.format(tankAmt));
  }
}
```

Screenshot Hasil



Penjelasan Permasalahan dan Solusi

Permasalahan:
Program dituntut untuk dapat menghitung jumlah galon cat yang dibutuhkan untuk menutupi berbagai bentuk bangun (seperti Sphere, Rectangle, dan Cylinder). Permasalahan utamanya adalah bagaimana membuat satu buah fungsi kalkulasi di dalam kelas Paint yang mampu memproses berbagai bangun tersebut secara universal, tanpa harus menulis ulang atau melakukan overloading method (misalnya membuat amount(Sphere s), amount(Rectangle r), dst) secara terpisah.

Solusi:
Masalah ini diselesaikan dengan memanfaatkan konsep Abstract Class dan Polymorphism via Inheritance (Polimorfisme melalui pewarisan).

- Kelas Shape dideklarasikan sebagai abstract class. Kelas abstrak ini tidak dapat diinstansiasikan menjadi objek , melainkan berfungsi untuk mengumpulkan atribut umum (seperti shapeName) dan mendeklarasikan abstract method area() yang tidak memiliki body method.
- Kelas Rectangle dan Cylinder dibuat sebagai concrete class yang extends kelas Shape. Kemudian kelas ini

mengimplementasikan secara spesifik abstract method area() yang diturunkan oleh superclass-nya. Pada Rectangle, area() mengembalikan nilai length * width. Pada Cylinder, area() mengembalikan Math.PI * radius * radius * height.

- Method amount cukup menerima satu parameter referensi superclass, yaitu Shape s. Pada program utama PaintThings.java, kita mengirimkan objek yang berbeda-beda (deck, bigBall, tank) ke dalam method tersebut. Saat perintah s.area() dieksekusi di dalam kelas Paint, Java menunda penentuan method hingga run-time dynamic binding. Java secara dinamis akan mengecek objek aktual apa yang sedang diproses (Rectangle, Sphere, atau Cylinder) dan langsung menjalankan rumus area() yang spesifik milik objek tersebut.

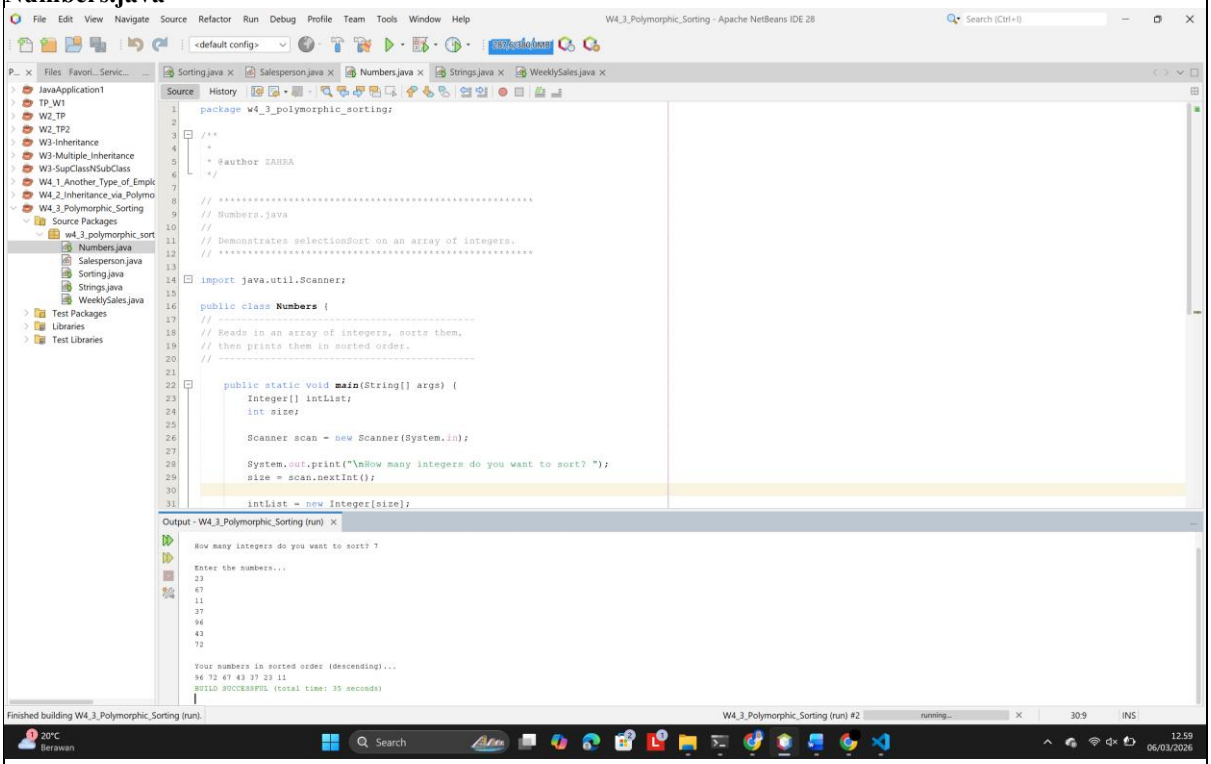
Penggunaan abstract class dan polymorphism memecahkan masalah kekakuan pada kode. Jika suatu saat sistem perlu menghitung cat untuk bangun ruang yang baru, kita tidak perlu mengubah kode di dalam kelas Paint atau membuat method amount() yang baru. Program secara dinamis akan memanggil rumus area() yang tepat saat run-time, membuat program menjadi sangat fleksibel.

Nama Teman Hal yang Dibantu (Opsional)

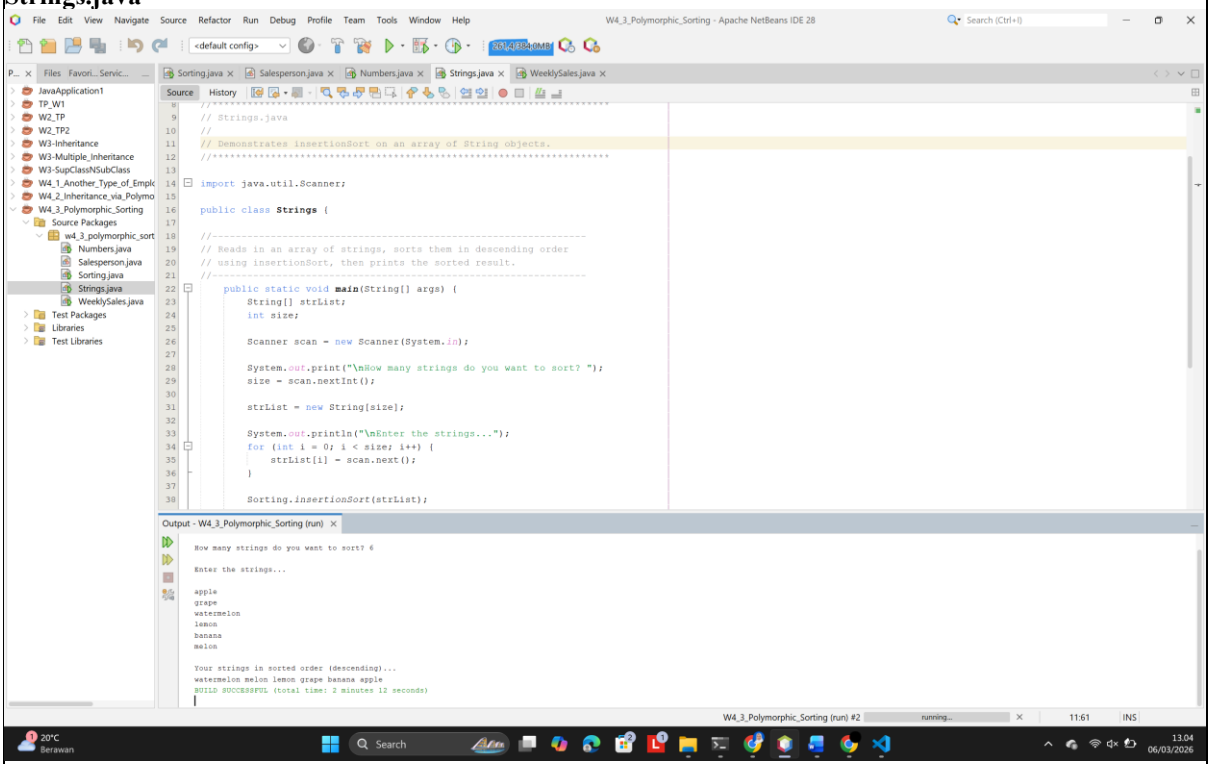
No. 3 Polymorphic Sorting

Soal Praktikum
Silahkan membaca soal pada file lampiran: 3-Polymorphic Sorting.pdf
Source Code
Screenshot Hasil

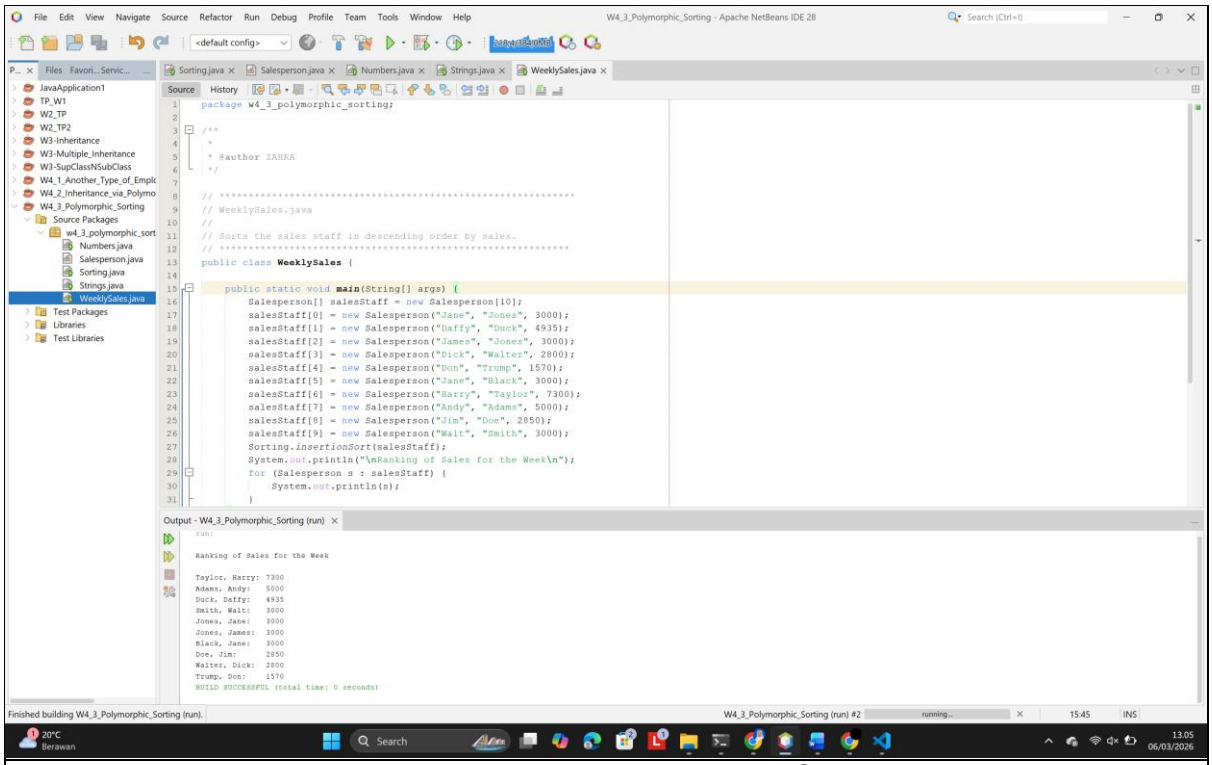
Numbers.java



Strings.java



WeeklySales.java



Penjelasan Permasalahan dan Solusi

Permasalahan:

Program Numbers.java mengalami error saat di-compile karena mencoba mengirimkan array bertipe data primitif (int[]) ke dalam method insertionSort yang secara spesifik meminta parameter array objek (Comparable[]). Kemudian, algoritma insertionSort bawaan perlu diubah agar mengurutkan data dari besar ke kecil (descending). Selain itu, kita harus merancang method compareTo pada kelas Salesperson untuk membandingkan angka penjualan, dan menggunakan nama sebagai penentu (tie-breaker) jika angka penjualannya sama.

Solusi:

- Pada Numbers.java, tipe deklarasi array diubah dari int[] menjadi kelas wrapper Integer[]. Hal ini diperlukan uuntuk menuntaskan masalah error type-mismatch karena Integer adalah sebuah objek yang memenuhi kontrak interface Comparable, dan Java secara otomatis membungkus nilai primitifnya melalui fitur autoboxing.
- Pada Sorting.java, logika perulangan while di dalam method insertionSort diubah tandanya dari < 0 menjadi > 0 (key.compareTo(list[position - 1]) > 0) dengan begitu sistem melakukan peletakan elemen terkecil ke sisi kanan.
- Pada Salesperson.java, method compareTo diimplementasikan dengan logika dasar (mengembalikan -1 jika totalSales lebih kecil dan 1 jika lebih besar), kemudian memanggil String.compareTo() pada lastName dan firstName jika menemukan nilai seri, sehingga menghindari kebingungan komparator double negative dan menghasilkan reverse alphabethical ketika nilai penjualannya seri.

Solusi ini menunjukkan penerapan konsep Polymorphism via Interfaces. Melalui interface Comparable, algoritma insertionSort diubah menjadi fungsi universal yang dapat digunakan kembali untuk memproses berbagai tipe objek yang berbeda. Solusi ini juga memperlihatkan pemisahan tanggung jawab dimana kelas Sorting untuk memanipulasi arah pergeseran algoritma menjadi descending, sementara kelas Salesperson difokuskan murni untuk mendefinisikan aturan nilai objeknya.

Nama Teman Hal yang Dibantu (Opsional)